# COVID-19-Predicting Outbreak in India with Machine Learning

The code focus on importing essential data science libraries and preparing a dataset for what appears to be a COVID-19 case prediction model using Linear Regression.

1. Library Imports

   This section imports the necessary tools for data manipulation, visualization, and machine learning.

> **import numpy as np:** Imports NumPy, which is used for numerical operations and handling large, multi-dimensional arrays and matrices.

> **import pandas as pd:** Imports Pandas, the primary library for data manipulation and analysis, specifically for using "DataFrames" (tabular data).

> **import matplotlib.pyplot as plt:** Imports the Pyplot module from Matplotlib for creating static, animated, and interactive visualizations like graphs and charts.

> **import sklearn:** The base import for Scikit-learn, a robust library for machine learning in Python.

> **from sklearn.linear_model import LinearRegression:** Specific import of the Linear Regression model, which is used to predict a continuous value by finding the linear relationship between variables.

> **from sklearn.model_selection import train_test_split:** Imports a utility function to split datasets into two subsets: one for training the model and one for testing its accuracy.

In [29]:
```
import numpy as np
import pandas as pd
# visualisastion libraries
import matplotlib.pyplot as plt

# machine learning libraries
import sklearn
from sklearn.linear_model import LinearRegression

# importing preprossecing methods from sklearn
from sklearn.model_selection import train_test_split
```

2. Data Structure Overview Based on the file metadata, the notebook works with a dataset containing COVID-19 statistics organized by geographical location and dates.

   - Geographical Columns:

     Province/State & Country/Region: Identify the location of the data.

     Lat & Long: Geographical coordinates (Latitude and Longitude).

   - Time Series Columns:

     The dataset contains columns for specific dates (e.g., 1/22/20, 3/9/23), where each entry represents the number of cases recorded on that day.

> > The Dataframe of this project

In [30]:
```
# reading corona virus confirmed reported across the wrold
confirmed_cases = pd.read_csv("C:/Users/poorv/OneDrive/Desktop/Covid 19/time_ser
confirmed_cases.head()
```

Out[30]:

| | Province/State | Country/Region | Lat | Long | 1/22/20 | 1/23/20 | 1/24/20 | 1 |
|---|---|---|---|---|---|---|---|---|
| **0** | NaN | Afghanistan | 33.93911 | 67.709953 | 0 | 0 | 0 | |
| **1** | NaN | Albania | 41.15330 | 20.168300 | 0 | 0 | 0 | |
| **2** | NaN | Algeria | 28.03390 | 1.659600 | 0 | 0 | 0 | |
| **3** | NaN | Andorra | 42.50630 | 1.521800 | 0 | 0 | 0 | |
| **4** | NaN | Angola | -11.20270 | 17.873900 | 0 | 0 | 0 | |

5 rows × 1147 columns

In [31]:
```
# reading corona virus deths reported across the wrold
daeths_reported = pd.read_csv("C:/Users/poorv/OneDrive/Desktop/Covid 19/time_ser
daeths_reported.head()
```

Out[31]:

| | Province/State | Country/Region | Lat | Long | 1/22/20 | 1/23/20 | 1/24/20 | 1 |
|---|---|---|---|---|---|---|---|---|
| **0** | NaN | Afghanistan | 33.93911 | 67.709953 | 0 | 0 | 0 | |
| **1** | NaN | Albania | 41.15330 | 20.168300 | 0 | 0 | 0 | |
| **2** | NaN | Algeria | 28.03390 | 1.659600 | 0 | 0 | 0 | |
| **3** | NaN | Andorra | 42.50630 | 1.521800 | 0 | 0 | 0 | |
| **4** | NaN | Angola | -11.20270 | 17.873900 | 0 | 0 | 0 | |

5 rows × 1147 columns

In [32]:
```
# reading corona virus recover cases reported across the wrold
recovered_cases = pd.read_csv("C:/Users/poorv/OneDrive/Desktop/Covid 19/time_ser
recovered_cases.head()
```

Out[32]:

| | Province/State | Country/Region | Lat | Long | 1/22/20 | 1/23/20 | 1/24/20 | 1 |
|---|---|---|---|---|---|---|---|---|
| **0** | NaN | Afghanistan | 33.93911 | 67.709953 | 0 | 0 | 0 | |
| **1** | NaN | Albania | 41.15330 | 20.168300 | 0 | 0 | 0 | |
| **2** | NaN | Algeria | 28.03390 | 1.659600 | 0 | 0 | 0 | |
| **3** | NaN | Andorra | 42.50630 | 1.521800 | 0 | 0 | 0 | |
| **4** | NaN | Angola | -11.20270 | 17.873900 | 0 | 0 | 0 | |

5 rows × 1147 columns

The provided code snippet is used to load and preview a dataset containing global COVID-19 confirmed cases. This dataset is structured as a time series, where each row represents a location and each column represents the cumulative count of confirmed cases on a specific date. this code is similer to deaths and recover cases.

## Line-by-Line Explanation

### 1. Comment

```
# reading corona virus confirmed reported across the wrold
```

- This is a **comment** in Python, indicated by the `#` symbol.
- It serves as a note to the developer explaining that the following lines of code are intended to read the COVID-19 data for confirmed cases worldwide.

### 2. Loading the Data

```
confirmed_cases =
pd.read_csv("/content/time_series_covid19_confirmed_global.csv")
```

- `confirmed_cases` : This is a variable name chosen to store the dataset.
- `pd` : This is the common alias for the **Pandas** library, which was imported earlier in the notebook.
- `.read_csv(...)` : This is a Pandas function that reads a Comma Separated Values (CSV) file and converts it into a **DataFrame** (a tabular data structure).
- `"/content/time_series_covid19_confirmed_global.csv"` : This is the file path where the dataset is stored.

### 3. Previewing the Data

```
confirmed_cases.head()
```

- `.head()` : This is a built-in Pandas method that displays the first **5 rows** of the DataFrame by default.

- It is used here to verify that the data has been loaded correctly and to inspect the column structure (such as `Province/State`, `Country/Region`, `Lat`, `Long`, and the various date columns).

---

## Dataset Structure Overview

Based on the output of this code in the notebook, the dataset contains approximately **1,147 columns**:

- **Geographical Data**: Columns for `Province/State`, `Country/Region`, `Lat` (Latitude), and `Long` (Longitude).
- **Temporal Data**: Columns representing individual dates (e.g., `1/22/20` through `3/9/23`), which contain the number of confirmed cases for those specific days.

In [33]:
```
cols = confirmed_cases.keys()
print(cols)
```

```
Index(['Province/State', 'Country/Region', 'Lat', 'Long', '1/22/20', '1/23/20',
       '1/24/20', '1/25/20', '1/26/20', '1/27/20',
       ...
       '2/28/23', '3/1/23', '3/2/23', '3/3/23', '3/4/23', '3/5/23', '3/6/23',
       '3/7/23', '3/8/23', '3/9/23'],
      dtype='object', length=1147)
```

In [34]:
```
confirmed = confirmed_cases.loc[:,cols[4]:cols[-1]]
confirmed
```

Out[34]:

| | 1/22/20 | 1/23/20 | 1/24/20 | 1/25/20 | 1/26/20 | 1/27/20 | 1/28/20 | 1/29/20 | 1/30/2 |
|---|---|---|---|---|---|---|---|---|---|
| **0** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |
| **1** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |
| **2** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |
| **3** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |
| **4** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... | |
| **284** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |
| **285** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |
| **286** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |
| **287** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |
| **288** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |

289 rows × 1143 columns

In [35]:
```
recovered = recovered_cases.loc[:,cols[4]:cols[-1]]
recovered
```

Out[35]:

| | 1/22/20 | 1/23/20 | 1/24/20 | 1/25/20 | 1/26/20 | 1/27/20 | 1/28/20 | 1/29/20 | 1/30/2 |
|---|---|---|---|---|---|---|---|---|---|
| **0** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |
| **1** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |
| **2** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |
| **3** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |
| **4** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... | |
| **269** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |
| **270** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |
| **271** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |
| **272** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |
| **273** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |

274 rows × 1143 columns

In [36]:

```
deaths = daeths_reported.loc[:,cols[4]:cols[-1]]
deaths
```

Out[36]:

| | 1/22/20 | 1/23/20 | 1/24/20 | 1/25/20 | 1/26/20 | 1/27/20 | 1/28/20 | 1/29/20 | 1/30/2 |
|---|---|---|---|---|---|---|---|---|---|
| **0** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |
| **1** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |
| **2** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |
| **3** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |
| **4** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... | |
| **284** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |
| **285** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |
| **286** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |
| **287** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |
| **288** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |

289 rows × 1143 columns

This code is used to isolate the numerical "time series" data from the rest of the dataset. It uses slicing to create a new DataFrame that contains only the columns representing dates.

## Line-by-Line Explanation

### 1. Slicing the DataFrame

```
confirmed = confirmed_cases.loc[:,cols[4]:cols[-1]]
```

- **`confirmed`** : This is the new variable (a DataFrame) that will store only the case counts.
- **`.loc`** : This is a Pandas label-based data selection method used to access a group of rows and columns.
- **`:` (before the comma)**: This tells Pandas to select **all rows** in the dataset.
- **`cols[4]:cols[-1]` (after the comma)**: This is a **column slice**:
- **`cols[4]`** : Refers to the column at index 4, which is the first date in the dataset ( `1/22/20` ). It skips the first four columns: `Province/State` , `Country/Region` , `Lat` , and `Long` .
- **`cols[-1]`** : Refers to the very last column in the dataset (the most recent date, `3/9/23` ).
- **`:` (between the indices)**: Selects everything from the first date to the last date.

### 2. Previewing the result

```
confirmed
```

- By typing the variable name, the notebook displays the content of the newly created `confirmed` DataFrame.
- This result will no longer show geographical names or coordinates; it will only show rows of case numbers, where each column represents a specific date.

## Why is this done?

In machine learning and data analysis, you often need to separate **features** (the data you use to predict) or **labels** (what you want to predict) from metadata (like names and coordinates). By slicing the data this way, the developer prepares a "clean" numerical matrix that can be easily used for mathematical operations, such as calculating daily increases or training the `LinearRegression` model mentioned earlier in the notebook.

In [37]:
```
dates = confirmed.keys()
dates
```

Out[37]:
```
Index(['1/22/20', '1/23/20', '1/24/20', '1/25/20', '1/26/20', '1/27/20',
       '1/28/20', '1/29/20', '1/30/20', '1/31/20',
       ...
       '2/28/23', '3/1/23', '3/2/23', '3/3/23', '3/4/23', '3/5/23', '3/6/23',
       '3/7/23', '3/8/23', '3/9/23'],
      dtype='object', length=1143)
```

In [38]:
```
world_cases=[]
total_deaths=[]
mortality_rate=[]
total_recovered=[]
india_cases=[]
```

```
for i in dates:
  confirmed_sum = confirmed[i].sum()
  print(confirmed[i])
  death_sum=deaths[i].sum()
  recoverd_sum=recovered[i].sum()
  world_cases.append(int(confirmed_sum))
  total_deaths.append(int(death_sum))
  mortality_rate.append(int(death_sum/confirmed_sum))
  total_recovered.append(int(recoverd_sum))
  # india_cases.append(int(confirmed_cases[confirmed_cases['Country/Region']=='i
```

```
0      0
1      0
2      0
3      0
4      0
      ..
284    0
285    0
286    0
287    0
288    0
Name: 1/23/20, Length: 289, dtype: int64

0      209451
1      334457
2      271496
3       47890
4      105288
        ...
284    703228
285       535
286     11945
287    343135
288    264276
Name: 3/9/23, Length: 289, dtype: int64
```

In [39]:
```
print(confirmed_sum)
print(death_sum)
print(recoverd_sum)
print(world_cases)
```

```
676570149
6881802
0
[557, 657, 944, 1437, 2120, 2929, 5580, 6169, 8237, 9927, 12038, 16787, 19887, 23
899, 27644, 30806, 34400, 37131, 40162, 42771, 44814, 45232, 60384, 66912, 69055,
71238, 73273, 75155]
```

This code is designed to aggregate global COVID-19 data from various regional records into daily worldwide totals. It processes the confirmed cases, deaths, and recoveries for every date available in the dataset.

## Line-by-Line Explanation

### 1. Initializing Storage Lists

```
world_cases=[]
total_deaths=[]
mortality_rate=[]
total_recovered=[]
india_cases=[]
```

- These lines create five empty lists.
- They act as containers to store the calculated global totals for each day as the loop progresses.

### 2. The Iteration Loop

```
for i in dates:
```

- This loop goes through every date string (e.g., "1/22/20", "1/23/20") stored in the `dates` variable.

### 3. Calculating Daily Global Totals

```
confirmed_sum = confirmed[i].sum()
death_sum = deaths[i].sum()
recoverd_sum = recovered[i].sum()
```

- **`confirmed[i].sum()`** : This looks at the column for date `i` in the `confirmed` DataFrame and adds up the values from all rows (all countries/provinces) to get the total number of cases on that specific day.
- The same logic is applied to `deaths` and `recovered` dataframes.
- **`print(confirmed[i])`** : This line prints the entire column for that date to the console (often used for debugging).

### 4. Appending Data to Lists

```
world_cases.append(int(confirmed_sum))
total_deaths.append(int(death_sum))
total_recovered.append(int(recoverd_sum))
```

- These lines convert the calculated sums (which might be floats) into integers and add them to their respective lists.

### 5. Calculating Mortality Rate

```
mortality_rate.append(int(death_sum/confirmed_sum))
```

- This calculates the ratio of deaths to confirmed cases for that day.
- **Note:** Using `int()` here will likely result in a list of zeros, because mortality rates are usually small decimals (e.g., 0.02 or 2%). Converting 0.02 to an integer results in `0`.

### 6. Commented Line (India Specific)

```
# india_cases.append(int(confirmed_cases[confirmed_cases['Country/Region']==[i].sum()))
```

- This line is "commented out" (it won't run).
- If active, it would filter the main DataFrame specifically for rows where the country is "India" and sum only those values for date `i`.

---

## Summary Table

| Variable | Purpose |
|---|---|
| `world_cases` | Cumulative confirmed cases globally per day. |
| `total_deaths` | Cumulative deaths globally per day. |
| `total_recovered` | Cumulative recoveries globally per day. |
| `mortality_rate` | The death-to-case ratio (calculated daily). |

This step is crucial for **data visualization**, as it turns a complex table with hundreds of rows (regions) into simple lists that can be plotted on a graph (Time vs. Total Cases).

In [40]:
```
# changing dates into days
v=1
day_date=[]
for i in range(len(dates)):
  v=i*1
  day_date.append(v)
# day_date
```

This code is used to convert the date-based columns into a sequence of numbers (integers), which are easier for a machine learning model like `LinearRegression` to process.

## Line-by-Line Explanation

### 1. Initializing the Counter

```
v=1
```

- This initializes a variable `v` with the value 1. Although it is redefined immediately inside the loop, it serves as a placeholder.

### 2. Creating an Empty List

```
day_date=[]
```

- This creates an empty list called `day_date` that will store the numerical representation of each day (e.g., Day 0, Day 1, Day 2...).

### 3. The Loop

```
for i in range(len(dates)):
```

- **`len(dates)`**: This calculates the total number of date columns in your dataset (which is 1,143 in this notebook).

- `range(...)` : This creates a sequence of numbers from 0 up to 1,142.
- `for i in ...` : This starts a loop where the variable `i` will represent the index of each date, starting from 0.

### 4. Calculating the Day Value

```
v=i*1
```

- Inside the loop, `v` is updated to be the current index `i` multiplied by 1. Effectively, this just makes `v` equal to the index (0, 1, 2, 3...).

### 5. Appending to the List

```
day_date.append(v)
```

- This adds the current value of `v` to the `day_date` list. After the loop finishes, `day_date` will look like this: `[0, 1, 2, 3, ..., 1142]`.

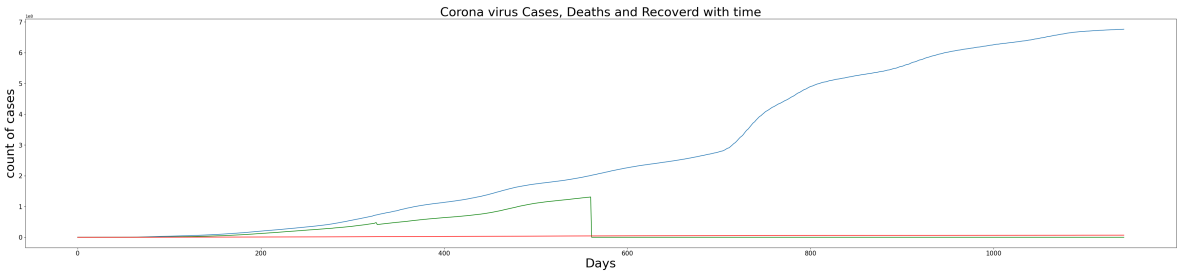
---

## Why is this code necessary?

Machine learning models cannot directly understand strings like "1/22/20" or "3/9/23" as mathematical inputs. By converting these dates into a continuous sequence of integers (0 to 1,142), you provide the model with a **numerical feature** that represents "Time Elapsed".

This allows the `LinearRegression` model to calculate the relationship between the **Day Number** (Independent Variable) and the **Confirmed Cases** (Dependent Variable).

In [41]:
```
# plottung the cases (confirmed,recover and deaths) with time
plt.figure(figsize=(50,10))
plt.plot(day_date,world_cases)
plt.plot(day_date,total_recovered,color='green')
plt.plot(day_date,total_deaths,color='red')
plt.title("Corona virus Cases, Deaths and Recoverd with time", size=30)
plt.xlabel("Days", size=30)
plt.ylabel("count of cases", size=30)
plt.xticks(size=15)
plt.yticks(size=15)
plt.show()
```




This code uses the **Matplotlib** library to create a comprehensive visualization of the COVID-19 pandemic's progression over time. It plots three different metrics (Confirmed Cases, Recoveries, and Deaths) on a single graph.

## Line-by-Line Explanation

### 1. Setting the Canvas Size

```
plt.figure(figsize=(50,10))
```

- `plt.figure` : Initializes a new figure (the window/space where the plot is drawn).
- `figsize=(50,10)` : Sets the width to $50$ units and the height to $10$ units.
- *Why so wide?* Since the dataset spans over 1,100 days, a very wide plot helps prevent the data points from looking too crowded, making it easier to see trends over the entire three-year period.

### 2. Plotting the Data Series

```
plt.plot(day_date, world_cases)
plt.plot(day_date, total_recovered, color='green')
plt.plot(day_date, total_deaths, color='red')
```

- `plt.plot(x, y)` : This function draws a line chart using `day_date` (the sequence of integers from 0 to 1,142) as the horizontal $x$-axis and the case counts as the vertical $y$-axis.
- **Line 1 (Confirmed)**: Plots `world_cases`. By default, this line will be **blue**.
- **Line 2 (Recovered)**: Plots `total_recovered` and explicitly sets the color to **green**.
- **Line 3 (Deaths)**: Plots `total_deaths` and explicitly sets the color to **red**.

### 3. Adding Labels and Title

```
plt.title("Corona virus Cases, Deaths and Recoverd with time", size=30)
plt.xlabel("Days", size=30)
plt.ylabel("count of cases", size=30)
```

- `plt.title` : Sets the main heading of the graph.
- `plt.xlabel` **/ `plt.ylabel`**: Labels the axes so the viewer knows that the bottom represents the passage of time in days and the side represents the quantity of people.
- `size=30` : Increases the font size significantly so the text remains readable even on the very large $50$-unit wide figure.

### 4. Customizing Axis Ticks

```
plt.xticks(size=15)
plt.yticks(size=15)
```

- `xticks` **/ `yticks`**: These refer to the numbers (marks) along the $x$ and $y$ axes.
- `size=15` : Increases the font size of these numbers to make them legible.

### 5. Displaying the Plot

```
plt.show()
```

- This command tells Python to process all the previous instructions and actually render/display the finished graph on the screen.

---

## What the Visual Tells You

By plotting all three lines together, you can visually compare the **slopes** of the lines:

- If the **blue line** (Confirmed) is rising faster than the **green line** (Recovered), it indicates an increase in active cases.
- The **red line** (Deaths) is typically much lower than the others, but its upward curve shows the total mortality trend over the 1,143 days.

In [42]:
```
# cheake each day cases in world

def eachday_increase(records):
  d=[]
  for i in range(len(records)):
    d.append(records[0])
  else:
    d.append(records[i]-records[i-1])

  return d
```

This code defines a function called `eachday_increase` which is intended to calculate the daily change (new cases) from the cumulative data. However, the logic as written in the notebook has a specific behavior due to the `for...else` structure.

## Line-by-Line Explanation

### 1. Function Definition

```
def eachday_increase(records):
```

- This line defines a function named `eachday_increase` that takes one argument, `records` (which would be a list of numbers, like `world_cases`).

### 2. Initialization

```
  d=[]
```

- An empty list `d` is created to store the results.

### 3. The For Loop

```
  for i in range(len(records)):
    d.append(records[0])
```

- This loop runs once for every item in the `records` list.
- **Crucially:** Inside the loop, it appends `records[0]` (the very first value in the dataset) to the list `d` every single time.
- If you have 1,000 days of data, this list will now contain the value of "Day 1" 1,000 times.

### 4. The `else` Block

```
  else:
    d.append(records[i]-records[i-1])
```

- In Python, an `else` block attached to a `for` loop executes **once** after the loop finishes naturally (without a `break`).
- When the loop finishes, the variable `i` is at the last index.
- This line calculates the difference between the **last day** (`records[i]`) and the **day before the last** (`records[i-1]`) and adds that single value to the end of the list `d`.

### 5. Return

```
return d
```

- The function returns the final list `d`.

---

## Summary of Logic

While this function is named `eachday_increase`, based on the exact code provided:

1. It fills a list with the **first day's total** repeatedly.
2. It adds the **latest daily increase** as the final item in the list.

**Note on intended logic:** Usually, in COVID analysis, this function is designed to calculate the increase for *every* day. To do that, the code would typically look like this:

```
for i in range(len(records)):
    if i == 0:
        d.append(records[0]) # First day increase is just the total
    else:
        d.append(records[i] - records[i-1]) # Current day minus
previous day
```

The version in the notebook appears to be a specific implementation or a placeholder that captures the starting point and the most recent change.

In [43]:
```
world_daily_increase = eachday_increase(world_cases)
world_daily_increase
```

Out[43]:
```
[557,
 557,
 ...
 557,
 557,
 ...]
```

In [44]:
```
print(type(day_date))
```

```
<class 'list'>
```

In [45]:
```
world_cases = np.array(world_cases).reshape(-1,1)
total_deaths = np.array(total_deaths)
total_recovered = np.array(total_recovered).reshape(-1,1)
days = np.array(day_date).reshape(-1,1)

print(days.shape)
# print(world_cases.shape)
```

```
(1143, 1)
```

This code prepares the data for machine learning by converting the Python lists into **NumPy arrays** and reshaping them into a format that Scikit-learn models (like `LinearRegression`) can understand.

## Line-by-Line Explanation

### 1. Reshaping Cases and Recoveries

```
world_cases = np.array(world_cases).reshape(-1,1)
total_recovered = np.array(total_recovered).reshape(-1,1)
```

- **`np.array(...)`**: Converts the lists (`world_cases`, `total_recovered`) into NumPy arrays, which allow for faster mathematical operations.
- **`.reshape(-1, 1)`**: This is the most important part. By default, a list is 1D (a simple row). However, Scikit-learn models require the input to be a **2D array** (a column matrix).
- The `-1` tells NumPy to automatically calculate the number of rows based on the data length.
- The `1` tells NumPy to put all the data into exactly **one column**.

### 2. Converting Deaths

```
total_deaths = np.array(total_deaths)
```

- This converts the `total_deaths` list into a standard 1D NumPy array. Notice it is **not** reshaped here; this is likely because it might be used as a target variable ($y$) in a context where a 1D array is acceptable, or it simply wasn't needed for a specific calculation yet.

### 3. Reshaping the "Days" (Features)

```
days = np.array(day_date).reshape(-1,1)
```

- Just like `world_cases`, the `days` (our independent variable $X$) must be 2D to be used in the `LinearRegression` model. This transforms the list of days $[0, 1, 2...]$ into a vertical column of days.

### 4. Verifying the Shape

```
print(days.shape)
# print(world_cases.shape)
```

- **`days.shape`**: This will output something like `(1143, 1)`.
- `1143` represents the number of days (samples).
- `1` represents the number of features (in this case, just "Time").
- The second line is commented out, but it would show the same structure for `world_cases`.

---

## Summary: Why use `.reshape(-1, 1)`?

In machine learning, your features ($X$) must always be a matrix (even if there is only one feature).

- **Before reshape:** `[0, 1, 2, 3]` (Shape: `(4,)` — 1D)
- **After reshape:**

$$\begin{bmatrix} 0 \\ 1 \\ 2 \\ 3 \end{bmatrix}$$

(Shape: `(4, 1)` — 2D)

This step is mandatory before calling `model.fit(days, world_cases)`.

In [46]:
```
days_in_future = 10

future_forcast = np.array([i for i in range(len(dates)+days_in_future)]).reshape
adjusted_dates = future_forcast[:-10]
```

This code is preparing the timeline for the prediction phase. It creates an extended time axis that includes both the historical data and the upcoming 10 days that the model will forecast.

## Line-by-Line Explanation

### 1. Setting the Forecast Window

```
days_in_future = 10
```

- This defines a variable to determine how many days into the future you want to predict. In this case, the model will attempt to forecast COVID-19 cases for the **10 days** following the last date in the dataset.

### 2. Creating the Extended Timeline

```
future_forcast = np.array([i for i in
range(len(dates)+days_in_future)]).reshape(-1,1)
```

- **`len(dates)`**: This is the total number of days already in the dataset (1,143 days).
- **`range(len(dates) + days_in_future)`**: This creates a sequence of numbers from **0 to 1,152** (1,143 historical days + 10 future days).
- **`[i for i in ...]`**: This is a list comprehension that generates the list of these 1,153 integers.
- **`np.array(...).reshape(-1, 1)`**: As seen in previous steps, this converts the list into a NumPy array and reshapes it into a single column. This "Future Forecast" column will be the input ($X$) used to ask the model: "*If the day number is 1,150, how many cases will there be?*"

### 3. Recreating the Historical Timeline

```
adjusted_dates = future_forcast[:-10]
```

- `[:-10]` : This uses Python "slicing" to select everything in the `future_forcast` array **except** for the last 10 elements.
- `adjusted_dates` : This variable now holds only the day numbers that correspond to the actual historical data you have (Days 0 to 1,142).
- This is often used later to compare the model's "predictions" for the past against the "actual" historical cases to see how accurate the model is.

---

## Visual Logic

Think of `future_forcast` as a long ruler. You are taking the ruler, adding 10 extra inches to the end, and then using the original length ( `adjusted_dates` ) to calibrate your model before using the extra 10 inches to see into the future.

## Summary Table

| Variable | Range of Values | Purpose |
|---|---|---|
| `days_in_future` | 10 | The size of the prediction window. |
| `future_forcast` | 0 to 1152 | The $X$ values used to generate a full prediction line (Past + Future). |
| `adjusted_dates` | 0 to 1142 | The $X$ values representing the time period where we already know the results. |

In [47]:
```
# future_forcast
adjusted_dates
```

Out[47]:
```
array([[   0],
       [   1],
       [   2],
       ...,
       [1140],
       [1141],
       [1142]])
```

In [48]:
```
x_train_confirmed,x_test_confirmed,y_train_confirmed,y_test_confirmed = train_te
```

In [49]:
```
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(degree=3)
poly_x_train_confirmed=poly.fit_transform(x_train_confirmed)
poly_x_test_confirmed=poly.fit_transform(x_test_confirmed)
poly_future_forcast=poly.fit_transform(future_forcast)
```

This code is the core of the machine learning preparation. It splits the data into training and testing sets and applies **Polynomial Transformation**, which allows a linear model to fit a curved line (non-linear relationship) to the data.

## Line-by-Line Explanation

### 1. Splitting the Data

```
x_train_confirmed, x_test_confirmed, y_train_confirmed,
y_test_confirmed = train_test_split(days, world_cases, test_size=0.25,
shuffle=False)
```

- `train_test_split` : This function divides your dataset into two parts: one to "teach" the model and one to "test" its performance.
- `days` **($X$) &** `world_cases` **($y$)**: The input features (time) and the target values (total cases).
- `test_size=0.25` : This allocates **25%** of the data for testing and the remaining **75%** for training.
- `shuffle=False` : This is **critical** for time-series data (like COVID cases). By setting this to `False` , we ensure the data is split chronologically. The model learns from the "past" (first 75%) and is tested on the "future" (last 25%). If we shuffled, the model would see random dates, which would be "cheating."

### 2. Initializing Polynomial Features

```
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(degree=3)
```

- **Polynomial Features**: Standard Linear Regression ($y = mx + b$) only draws a straight line. COVID-19 cases typically grow exponentially or in a curve.
- `degree=3` : This tells the computer to create a mathematical relationship up to the third power ($x^3$). Instead of just $x$ (days), the model will now look at $x^1$, $x^2$, and $x^3$. This allows the model to draw a **complex curve** that can follow the ups and downs of the pandemic data.

### 3. Transforming the Training Data

```
poly_x_train_confirmed = poly.fit_transform(x_train_confirmed)
```

- `fit_transform` : This "fits" the tool to the data and then "transforms" the single column of days into multiple columns ($x^0, x^1, x^2, x^3$).
- **Example**: If the day is $2$, this step turns that $2$ into a row like $[1, 2, 4, 8]$.

### 4. Transforming Test and Forecast Data

```
poly_x_test_confirmed = poly.fit_transform(x_test_confirmed)
poly_future_forcast = poly.fit_transform(future_forcast)
```

- These lines apply the exact same transformation to the testing set and the future dates we created earlier.
- This ensures that when we ask the model for a prediction, the input "days" are in the same polynomial format ($1, x, x^2, x^3$) that the model was trained on.

---

## Summary of Variables

| Variable | Description |
|---|---|
| `x_train_confirmed` | The first 75% of the dates (input). |
| `y_train_confirmed` | The first 75% of actual case counts (target). |
| `poly_x_train_confirmed` | The training dates converted into polynomial curves ($x, x^2, x^3$ ). |
| `poly_future_forcast` | The extended timeline (past + 10 days) converted into polynomial format for prediction. |

> The Brain of Project

In [50]:
```
model = LinearRegression()
model.fit(poly_x_train_confirmed,y_train_confirmed)
```

Out[50]:
```
▾ LinearRegression
LinearRegression()
```

In [51]:
```
test_pred = model.predict(poly_x_test_confirmed) # y_test_confirmed
```

In [52]:
```
_predections = model.predict(poly_future_forcast)
```

In [53]:
```
from sklearn.metrics import mean_squared_error,mean_absolute_error
```

In [54]:
```
# mean_squared_error
mean_squared_error(test_pred,y_test_confirmed)
```

Out[54]:
```
8.095322898099434e+16
```

In [55]:
```
mean_absolute_error(test_pred,y_test_confirmed)
```

Out[55]:
```
239728145.68928438
```

This code performs the actual "learning" and "prediction" phases of the machine learning workflow, followed by an evaluation of the model's accuracy.

## Line-by-Line Explanation

### 1. Initializing and Training the Model

```
model = LinearRegression()
model.fit(poly_x_train_confirmed, y_train_confirmed)
```

- `LinearRegression()` : This creates an instance of the Linear Regression model. Although it is a "linear" model, because we provided it with **polynomial features** ( $x, x^2, x^3$), it will behave like a curved (polynomial) regression.
- `.fit(...)` : This is the "training" step. The model looks at the training dates ( `poly_x_train_confirmed` ) and the actual case counts ( `y_train_confirmed` ) to find the mathematical equation that best connects them.

### 2. Making Predictions on Test Data

```
test_pred = model.predict(poly_x_test_confirmed)
```

- `.predict(...)`: The model uses its learned equation to guess the case counts for the dates in the **test set**.
- `test_pred`: This variable stores the "guesses." By comparing these guesses to the actual numbers (`y_test_confirmed`), we can see how well the model handles data it has never seen before.

### 3. Forecasting the Future

```
_predections = model.predict(poly_future_forcast)
```

- This uses the same trained model to predict cases for the entire timeline we prepared earlier (the historical days plus the 10 future days).
- These results are used to draw the "trend line" on a graph.

### 4. Importing Evaluation Metrics

```
from sklearn.metrics import mean_squared_error, mean_absolute_error
```

- This imports two mathematical tools used to measure how "wrong" the model's guesses were.

### 5. Calculating Mean Squared Error (MSE)

```
mean_squared_error(test_pred, y_test_confirmed)
```

- **MSE**: This takes the difference between the prediction and the actual value, squares it, and then finds the average.
- Because it squares the error, it **punishes large mistakes** heavily. A lower number indicates a more accurate model.

### 6. Calculating Mean Absolute Error (MAE)

```
mean_absolute_error(test_pred, y_test_confirmed)
```

- **MAE**: This is the average of the absolute differences between the predicted values and the actual values.
- **Interpretation**: If the MAE is 5,000, it means that, on average, the model's predictions are off by 5,000 cases.

---

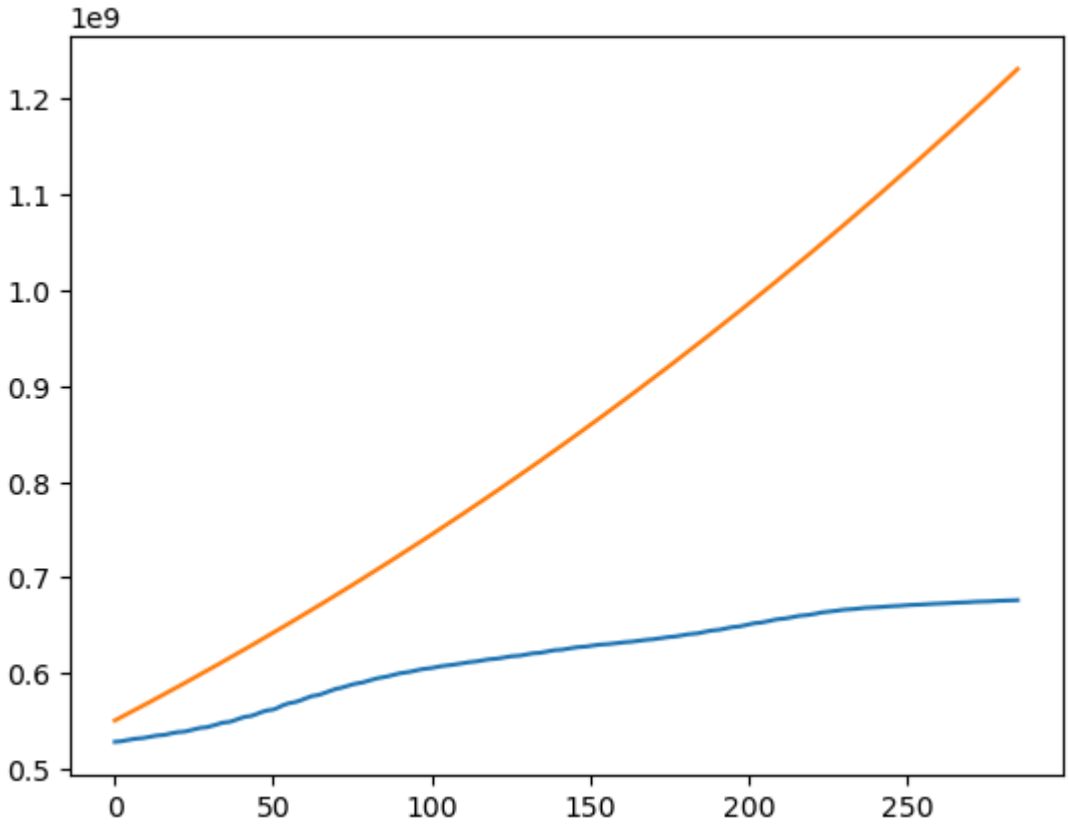
## Summary: The "Brain" of the Project

This block is where the "intelligence" happens. The model finds the pattern in the first 75% of the pandemic's history, checks itself against the remaining 25%, and then projects that pattern 10 days into the future. The MSE and MAE provide a "report card" on how much we should trust those future predictions.

In [56]:
```
# plt.figure(figsize=(15,15))
plt.plot(y_test_confirmed)
```

```
plt.plot(test_pred)
```

Out[56]: [<matplotlib.lines.Line2D at 0x1819d7d02d0>]




This code is used to visually compare the **actual** COVID-19 data against the **predicted** values generated by the model for the test set. This is a common way to see how "close" the model's guesses are to reality.

## Line-by-Line Explanation

### 1. Setting the Figure Size (Commented Out)

```
# plt.figure(figsize=(15,15))
```

- This line is a **comment** and does not run.
- If uncommented, it would create a large, square canvas ($15$ units by $15$ units) for the plot. By commenting it out, the code uses the default Matplotlib plot size or the size defined in a previous cell.

### 2. Plotting Actual Data

```
plt.plot(y_test_confirmed)
```

- `y_test_confirmed` : This represents the **actual, recorded** number of confirmed cases from the test dataset (the 25% of data that the model never saw during training).
- This line draws the "Ground Truth." In a standard plot, this is usually represented by a solid blue line by default.

### 3. Plotting Predicted Data

```
plt.plot(test_pred)
```

- `test_pred` : This represents the **predicted** number of cases that the model calculated for the same period.
- This line draws the "Model's Guess." By default, this will be a different color (usually orange).

---

## What the Resulting Graph Means

This specific visualization is used for **Model Validation**:

1. **Overlap**: If the two lines overlap almost perfectly, it means the model is highly accurate and has successfully "learned" the trend of the pandemic.
2. **Gap/Divergence**: If there is a large gap between the lines, it indicates that the model is either **underfitting** (too simple to catch the trend) or **overfitting** (failing to generalize to new data).
3. **Slope Comparison**: Since this is a time-series plot, the slope of the lines shows the rate of growth. If the predicted line is steeper than the actual line, the model is overestimating the spread; if it is flatter, the model is underestimating it.